

# Rodrigo Teixeira

Marketing Analytics Director · Data Engineer · Quant Builder

RFQ RESPONSE · CIPHERSALT LEAN POC

rodtex.dev  
github.com/strod  
linkedin.com/in/strodrigo  
São Paulo · Remote · EN/PT/ES

Eight years building **weighted scoring engines, MarTech data pipelines, and CMO-grade dashboards** at the intersection of Python, SQL, and cloud. Currently leading data engineering and data science at **Filterbuy** (US e-commerce), with hands-on Amazon Marketing Cloud, optimization-based "Moneyball" logic, and multi-LLM agentic UI experience. This portfolio maps my work directly to the CipherSalt POC's three-tier data flow, Salt Score engine, and agentic dashboard.

## RFQ FIT MAP · WHERE I'VE DONE THIS BEFORE

### CIPHERSALT NEEDS

17 metrics → weighted "Salt Score" with swappable Moneyball logic



Built **AIRE** at MINT — a multi-channel scoring & budget reallocation engine using linear optimization, lifted ROI **25%+**. Comfortable owning both the formula and the audit trail.

### CIPHERSALT NEEDS

AMC SQL extraction + CSV join engine (three-tier data)



Run **Filterbuy's Amazon Advertising stack end-to-end** — AMC-adjacent SQL, Reporting API automation, and a Lambda-based **Amazon Bids Optimizer** that joins procurement-style and performance data daily.

### CIPHERSALT NEEDS

Python data bridge → Next.js / RSC / Vercel AI SDK



Shipped **multi-LLM applications with generative UI** (Adagio Chatbot, AI Media Planner) using FastAPI back-ends streaming structured component output to React front-ends.

### CIPHERSALT NEEDS

Investor-ready demo with tick-and-tie auditability



Delivered **\$100M+ GM Marketing Mix Model** at WMcCann — every recommended dollar traceable from raw spend → coefficient → board slide. Same audit discipline applied here.

## CASE STUDIES · PATTERN-MATCH TO THE POC

## AIRE — AI Recommendation Engine (Weighted Scoring + Optimization Dial Analogue)

MINT · 2020–2022 · Lead Algorithm Engineer

SALT SCORE PARALLEL

Built the core engine MINT sold to enterprise advertisers: ingest performance data across digital channels, fit a constrained linear optimization model, and recommend a reallocated budget vector that maximizes ROI under business rules (min/max per channel, frequency caps, brand-safety floors).

The interaction model is the same one CipherSalt's **Optimization Dial** needs: **user changes a constraint → backend recomputes → UI re-ranks instantly**. I own that pattern from formula design to streaming UI update. Result: **+25% ROI lift** across deployed accounts, plus a Markowitz-style Predictive Planning Algorithm productionized alongside it.

Python

scipy.optimize

Linear Programming

FastAPI

GCP / Cloud Run

React Frontend

## Filterbuy Amazon Bids Optimizer — Production AMC-Adjacent Pipeline

Filterbuy · 2024–Present · Director, Marketing Analytics

AMC + CSV JOIN PARALLEL

Designed and deployed the bid-optimization engine that runs Filterbuy's Amazon Advertising spend. AWS Lambda pulls Amazon Reporting API data, joins it against internal procurement / margin CSVs (the same shape as your Direct + Synthesized data), applies a weighted scoring model per ASIN, and writes back recommended bids — all with full reproducibility and a tick-and-tie audit log per run.

I navigate AMC, write the SQL, manage the data lake (S3 + Redshift), and own the delivery layer end-to-end — exactly the autonomy your RFQ requires under "navigate the AMC environment without oversight."

Amazon Marketing Cloud

AWS Lambda

Redshift

S3

Python

SQL

## Adagio Chatbot & AI Media Planner — Agentic UI with Streaming Components

Personal / Adagio · 2024 · Architect & Builder

STREAMUI / AGENTIC UX PARALLEL

Two production-grade LLM apps where the agent does not return prose — it **renders structured components**. Adagio Chatbot answers tea-shop product queries by streaming a recommendation card (image, fit reasoning, pairing). AI Media Planner ingests a brief and streams a budget-allocation grid the planner can edit in place.

This is the exact pattern your "**agent explains the Salt Score**" requirement calls for: the model reads the Scorecard, decides which 3–5 metrics dominate, and renders a custom explanation component on the fly via Vercel AI SDK streamUI.

Vercel AI SDK

Next.js / RSC

FastAPI

Multi-LLM (OpenAI / Anthropic)

Embeddings + Vector Store

## GM Marketing Mix Model — \$100M+ Audit-Grade Recommendation

WMcCann (McCann Worldgroup) · 2019 · Data Analytics Manager

### TICK-AND-TIE DISCIPLINE

Led an MMM study sizing General Motors' annual marketing budget. The model touched a hundred million dollars of spend and was reviewed at executive level — every coefficient, transformation, and recommended reallocation had to be traceable on demand. Built the data pipeline, the model, and the reporting layer.

That audit muscle is what I'd bring to your milestone reviews: a written tick-and-tie sheet showing 3–5 suppliers traced raw → derived → weighted → final, so you can point at any number and I can point at the line of code.

Marketing Mix Modeling    BigQuery    Python / R    Executive Reporting

## Filterbuy MMM (Google Meridian) + Markov Attribution — Current Live System

Filterbuy · 2024–Present

### MULTI-SOURCE DATA UNIFICATION

Operating MMM via Google Meridian and a Markov-chain attribution model in parallel, both fed by ETL pipelines consolidating Amazon, TikTok, Google Ads, Pipedrive CRM, and web analytics into a unified Redshift → BigQuery layer. RFM segmentation, predictive churn, and continuous A/B testing run on top.

Same fundamental challenge as CipherSalt's **Join engine**: heterogeneous source systems, conflicting keys, missing fields, and one supplier-grain (or customer-grain) view at the end.

Google Meridian    Markov Attribution    Redshift / BigQuery    DBT    Python    Looker / QuickSight

**Data Foundation & Salt Score Engine****M1**

30% complete · 30% payment

*Get raw inputs out of AMC, unify the three-tier data structure, ship a working "black box" Python scorer on real CSVs.*

- ▶ **AMC extraction pack** — documented SQL queries pulling Direct Data (Impressions, Deterministic Reach, Conversions) plus inputs for Derived Data (Path Depth, Revenue Velocity, New-to-Brand). Versioned CSV outputs.
- ▶ **Three-tier data dictionary** — one-page schema mapping each of the 17 metrics to Direct / Derived / Synthesized origin, with formula + source field for every Derived metric.
- ▶ **Join engine (Python)** — ingests Procurement and Performance CSVs, normalizes supplier keys, resolves duplicates → single supplier-level dataframe ready for scoring.
- ▶ **Salt Score v1 script** — parameterized weighting engine, Moneyball rules in a swappable config file, outputs a per-supplier Scorecard CSV.
- ▶ **QA tick-and-tie sheet** — 3–5 suppliers traced raw AMC field → derived metric → weighted contribution → final Salt Score, end-to-end.

**ACCEPTANCE** `python score.py` on provided CSVs produces a Scorecard whose top/bottom suppliers pass the founder's smell test; any score traceable in under a minute.

**Integrated POC · CSV-to-UI Pipeline & Optimization Dial****M2**

70% complete · 40% payment

*Wire the Python engine to a Next.js front-end via React Server Components. Sub-second dial response, real numbers behind every visual.*

- ▶ **Data Bridge (FastAPI)** — two surfaces: batch pre-compute path for steady-state views + on-demand recompute endpoint for dial interactions.
- ▶ **Next.js dashboard skeleton (RSC-first)** — v0.app starting layout cleaned up to production: typography system, spacing scale, color tokens, component primitives. Server Components fetch & aggregate; Client Components reserved for interactivity.
- ▶ **Optimization Dial (live)** — drag re-weights the Moneyball formula → Python recomputes → UI updates with optimistic streaming. Target: **<800 ms perceived latency** at POC volumes.
- ▶ **Situation Room v1** — read-only view of top movers, supplier scorecards, and breakdown of any Salt Score into its 17 contributing metrics.
- ▶ **Architecture map** — one-page diagram tracing dial-turn → recalc → render through every layer, with file paths and function names.

**ACCEPTANCE** Founder opens dashboard, turns dial, sees suppliers re-rank in near real time, clicks any score to see its formula breakdown sourced from real CSV values.

## Agentic UI, Demo Polish & Handover

M3

100% complete • 30% payment

*Take the working POC from "engineer-grade" to "investor-ready." Layer in the agent, harden, document, hand over.*

- **Vercel AI SDK / `streamUI` integration** — agent reads the Scorecard and renders custom components on the fly to *explain* a Salt Score (e.g. "Why is Supplier X #3?" → streamed component showing dominant weighted metrics, not prose).
- **Situation Room v2** — narrative views the CMO can present from: anomaly callouts, period-over-period deltas, what-if traces tied to dial movements.
- **End-to-end QA pass** — edge-case sweep (empty CSVs, missing joins, extreme weights), unit tests on the scoring module, regression check (same input → same Salt Score).
- **Demo script & founder run-book** — exact flow for the investor demo + short technical map (where formulas live, how to swap weights, how to refresh AMC data) so the founder is not dependent on me post-handover.
- **Final handover bundle** — Python repo + Next.js repo + Figma reference + data dictionary + architecture map. Work-for-hire IP transfer per Section 1.

**ACCEPTANCE** Founder runs the investor demo unaided; agent narrates Salt Scores convincingly; all 17 metrics tick-and-tie; repo is clean, documented, reproducible.

**Q1 · v0.app handoff & design refinement**

Treat v0 as a **layout sketch**, not a codebase. Pass 1: extract typography, spacing, color tokens into a design system. Pass 2: replace ad-hoc components with composable primitives. Pass 3: wire RSC data, strip the client-side fetch boilerplate v0 over-generates. Result: bespoke, premium feel, no AI-generated tells.

**Q2 · RSC & sub-second "Financial Terminal" feel**

FastAPI service exposes **two surfaces**: batch pre-compute (steady views) + on-demand recompute (dial). Server Components handle CSV aggregation server-side; Client Components only where interaction demands it. Pre-compute the Scorecard at ingest → dial stays under **800 ms** at POC volumes.

**Q3 · CSV-to-UI pipeline (dial → Python → render)**

User turns dial → Client Component fires server action with new weights → FastAPI recomputes Salt Score against the in-memory CSV pool → returns updated Scorecard rows → RSC streams re-rendered Situation Room with optimistic UI. Every step instrumented for audit.

**Q4 · Vercel AI SDK / streamUI for agent explanations**

Agent reads the Scorecard, identifies the top weighted contributors per supplier, and uses `streamUI` to render a custom explanation component (bar of metric weights, deltas vs peer set, narrative caption). No prose walls — every number renders as a structured, on-brand element.

**Q5 · GUI tooling beyond v0.app**

v0 + Next.js stays the backbone for the agentic feel. Figma added only as a design-system reference layer. Streamlit is rejected — fast to ship but won't deliver terminal-grade response or bespoke aesthetic the brief calls for.

**Q6 · QA & tick-and-tie**

Every milestone review ships with a written audit sheet: 3–5 suppliers traced raw AMC field → derived metric → weighted contribution → final Salt Score. Unit tests on the scoring module, edge-case sweep before founder review, regression check that identical input yields identical score.

## STACK I'D USE FOR THIS POC

**DATA & BACKEND**

Python 3.11+  
FastAPI  
pandas / polars  
scipy.optimize  
pydantic v2  
pytest

**FRONTEND & AGENT**

Next.js 14+ (App Router)  
React Server Components  
Vercel AI SDK · streamUI  
Tailwind + shadcn/ui  
v0.app (refined)  
TypeScript

**DATA SOURCES & OPS**

Amazon Marketing Cloud  
CSV data lake  
SQL (AMC dialect)  
Vercel deploy target  
GitHub + Actions  
Figma (reference only)

**Ready to start the AMC walkthrough.** Detailed milestones (30 / 70 / 100 split per Section 3) and Section 6 written confirmations have been submitted alongside this portfolio. Happy to do a 30-minute architecture call before you commit.

rodstex@gmail.com  
+55 11 93021-2911  
rodtex.dev